Skill 11

```
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

char history[MAX_COMMANDS][MAX_LENGTH];
int count = 0;
int next = 0;

void add_command(const char *command)
{
   strncpy(history[next], command, MAX_LENGTH - 1);
   history[next][MAX_LENGTH - 1] = '\0';

   next = (next + 1) % MAX_COMMANDS;

   if (count < MAX_COMMANDS)
      count++;

   printf("Stored: %s\n", command);
}

void display_history(void)
{
   printf("\n--- Command History ---\n");
```

```
    if (count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    int start = (next - count + MAX_COMMANDS) % MAX_COMMANDS;

    for (int i = 0; i < count; i++)
    {
        int index = (start + i) % MAX_COMMANDS;
        printf("%d. %s\n", i + 1, history[index]);
    }
}

void retrieve_command(int position)
{
    if (position < 1 || position > count)
    {
        printf("Invalid history position.\n");
        return;
    }

    int start = (next - count + MAX_COMMANDS) % MAX_COMMANDS;
    int index = (start + position - 1) % MAX_COMMANDS;

    printf("Command at position %d: %s\n",
           position, history[index]);
```

```
}

int main(void)
{
    add_command("ls");
    add_command("pwd");
    add_command("date");
    add_command("whoami");
    add_command("ls -l");

    display_history();

    printf("\nAdding command beyond capacity:\n");
    add_command("gcc program.c -o program");

    display_history();

    printf("\nRetrieving command:\n");
    retrieve_command(2);

    printf("\nTesting invalid position:\n");
    retrieve_command(10);

    printf("\nHistory consistency check:\n");

    if (count >= 0 && count <= MAX_COMMANDS &&
        next >= 0 && next < MAX_COMMANDS)
        printf("History buffer is consistent.\n");
```

```
    else
        printf("History buffer is inconsistent.\n");

    return 0;
}
```

```
snigdha@Snigdha:~$ cd ~/quotes_practical
snigdha@Snigdha:~/quotes_practical$ nano history.c
snigdha@Snigdha:~/quotes_practical$ gcc history.c -o history

snigdha@Snigdha:~/quotes_practical$
snigdha@Snigdha:~/quotes_practical$ ./history
Stored: ls
Stored: pwd
Stored: date
Stored: whoami
Stored: ls -l

--- Command History ---
1. ls
2. pwd
3. date
4. whoami
5. ls -l

Adding command beyond capacity:
Stored: gcc program.c -o program

--- Command History ---
1. pwd
2. date
3. whoami
4. ls -l
5. gcc program.c -o program

Retrieving command:
Command at position 2: date

Testing invalid position:
Invalid history position.

History consistency check:
History buffer is consistent.
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 4
#define MAX_ARGS 10

struct Command
{
   char *args[MAX_ARGS];
};

void execute_pipeline(struct Command commands[], int n)
{
   int pipefd[2 * (MAX_COMMANDS - 1)];
   pid_t pids[MAX_COMMANDS];

   if (n < 1 || n > MAX_COMMANDS)
   {
      fprintf(stderr, "Invalid number of commands.\n");
      return;
   }

   /* Create all required pipes before starting children. */
   for (int i = 0; i < n - 1; i++)
   {
      if (pipe(&pipefd[2 * i]) == -1)
      {
```

```
      perror("pipe");
      exit(EXIT_FAILURE);
    }
}

for (int i = 0; i < n; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
      perror("fork");
      exit(EXIT_FAILURE);
    }

    if (pids[i] == 0)
    {
      /* Read from the previous command's pipe. */
      if (i > 0)
      {
        if (dup2(pipefd[2 * (i - 1)], STDIN_FILENO) == -1)
        {
          perror("dup2 input");
          _exit(1);
        }
      }

      /* Write to the next command's pipe. */
```

```
        if (i < n - 1)
        {
            if (dup2(pipefd[2 * i + 1], STDOUT_FILENO) == -1)
            {
                perror("dup2 output");
                _exit(1);
            }
        }

        /* Close all original pipe descriptors. */
        for (int j = 0; j < 2 * (n - 1); j++)
            close(pipefd[j]);

        execvp(commands[i].args[0], commands[i].args);

        perror(commands[i].args[0]);
        _exit(127);
    }
  }

  /* Parent does not need the pipe descriptors. */
  for (int i = 0; i < 2 * (n - 1); i++)
    close(pipefd[i]);

  /* Wait for all child processes. */
  for (int i = 0; i < n; i++)
    waitpid(pids[i], NULL, 0);
}
```

```
int main(void)
{
    struct Command commands[MAX_COMMANDS] = {
        {{"printf", "apple\nbanana\napple\n", NULL}},
        {{"sort", NULL}},
        {{"uniq", NULL}}
    };

    int n = 3;

    printf("Pipeline structure:\n");
    printf("Command 1: printf\n");
    printf("Command 2: sort\n");
    printf("Command 3: uniq\n");
    printf("\nPipeline output:\n");

    fflush(stdout);
    execute_pipeline(commands, n);

    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ nano pipeline.c
snigdha@Snigdha:~/quotes_practical$ gcc pipeline.c -o pipeline
snigdha@Snigdha:~/quotes_practical$ ./pipeline
Pipeline structure:
Command 1: printf
Command 2: sort
Command 3: uniq

Pipeline output:
apple
banana
snigdha@Snigdha:~/quotes_practical$
```